DAW 5 – JAVASCRIPT

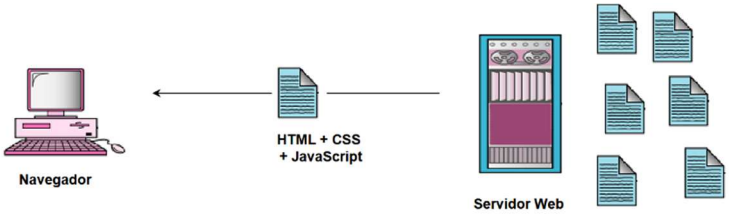
INTRODUCCIÓN A JAVASCRIPT

- JAVASCRIPT (JS) es un lenguaje de **programación interpretado** creado en 1995 por Brendan Eich para añadir interactividad a Netscape Navigator 2.0.
  - ↳ El **nombre original** fue **Mocha**, siendo rebautizado posteriormente mediante acuerdo con Sun Microsystems.
  - 🕑 En 1997 comenzó el **proceso de estandarización**, primero en **ECMA** (European Computer Manufacturers Association) con el nombre **ECMAScript** y posteriormente también **estándar ISO** (International Organization for Standardization).

INTEGRACIÓN CON HTML

- Con la aparición de JS se introduce el concepto de DHTML, **html dinámico**.
- JS aparece **embebido** en el código **html**, siendo **interpretado** (NO compilado) por el **navegador al descargar la página**.
  - ↳ En la actualidad se puede ejecutar JS en el lado servidor mediante el entorno de ejecución **Node.js** (no lo trataremos).

  Se pude incluir de dos maneras:
  - Colocando el código **directamente** como **contenido** de la etiqueta <script>.
  - Incluyendo el código como **página interna o externa** en un fichero **.js** especificado en <script src="*codigo*.js"> para **invocarlo** a lo **largo de la página**.
  - Con el fin de **mejorar la carga** de la página, se aconseja ponerlo al final, **antes de</body>**.



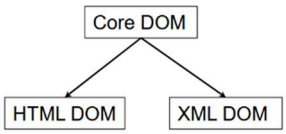
Navegador	Script Engine
Apple Safari	Nitro
Google Chrome	V8
Microsoft IE	Chakra
Mozilla Firefox	JägerMonkey
Opera	Carakan

OTROS LENGUAJES DE SCRIPT

- Microsoft desarrolló dos versiones propias para el Internet Explorer:
  - **VBScript** → **incompatible** con navegadores **diferentes a Internet Explorer** y con sintaxis basada en el **Visual Basic**.
  - **JScript** → versión de **ECMAScript** similar al **JavaScript de Netscape**, pero con algunas diferencias en el **modelo de objetos**.
    - ➢ La **diferencia en el modelo de objetos** entre JScript y JavaScript produjo **incompatibilidades**, por lo que W3C definió el **HTML DOM**, para aunar criterios.

HTML DOM

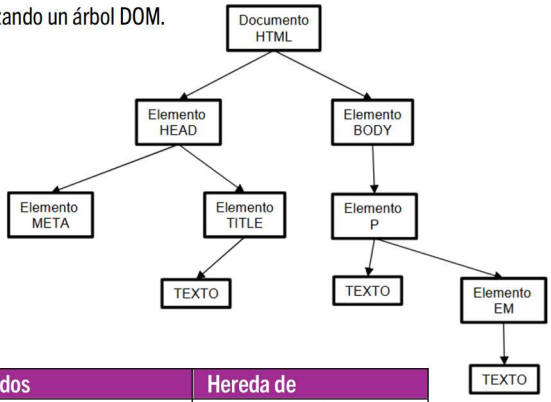
- El DOM (Document Object Model) es una API creada por **W3C** para definir un **mecanismo estándar de acceso a documentos html**, xml y svg permitiendo de esta manera que programas y scripts puedan **modificar** su **contenido**, **estructura** y/o **estilo**. Tiene 3 partes:
  - **Core** DOM → modelo estándar para cualquier documento estructurado.
  - **XML** DOM → modelo estándar para cualquier documento xml.
  - **HTML** DOM → modelo estándar para cualquier documento html (nos centraremos en este).



- Una página html está formada por múltiples etiquetas anidadas, formando un **árbol de etiquetas** conocido como ÁRBOL DOM (o simplemente DOM).
  - La **transformación automática** de la página en árbol sigue las siguientes reglas:
    - **Cada etiqueta** se transforma en **dos nodos** { **elemento** (objeto del DOM creado a partir de la etiqueta). nodo de **texto** (**hijo** del nodo de etiqueta).
    - Las **etiquetas hijas** de otras se transforman en nodos **hijo del nodo de la propia etiqueta padre**.

- La forma de **acceder al DOM** es a través de un objeto JS llamado **document**, que representa la página utilizando un árbol DOM.
  - document es una **propiedad** del objeto **window**, que representa al navegador.
  - document es una **instancia** de la clase **HTMLDocument**.

```
<html>
  <head>
    <meta ------------------ />
    <title> --------------------- </title>
  </head>
  <body>
    <p> ------ <em> ----- </em></p>
  </body>
</html>
```



INTERFACES DOM

Interfaz DOM	Descripción	Propiedades	Métodos	Hereda de
Document	Representa el **documento completo**.	…	….	Node
Element	Representa **cualquier etiqueta** del documento.	…	…	Node
HTMLDocument	Representa el **documento** html completo.			Document
HTMLElement	Representa **cualquier etiqueta html** del documento			Element
Attr	Representa **cualquier atributo** de las etiquetas.	name value specified		Node
Text	Representa los **nodos de texto** del documento.	data length	appendData() deleteData() insertData() replaceData()	Node
Comment	Representa los **comentarios** del documento.			Node
Style	Representa los **estilos** asociados a las etiquetas del documento.	cssText length parentRule	getPropertyPriority() getPropertyValue() removeProperty() setProperty()	CSSStyleDeclaration
Event	Representa los **eventos** asociados a las etiquetas del documento.	…	….	Object

Interfaz DOM	Descripción	Retornado por	Propiedades	Métodos
HTMLCollection	Lista de etiquetas html.	Métodos JS de acceso a etiquetas.	length	item(*index*), namedItem(*id*)
NodeList	Lista de objetos nodo del documento.	Métodos JS de acceso a nodos y etiquetas.	length	entries(),forEach(), item(*index*), keys(), values()
NamedNodeMap	Colección de atributos de etiquetas.	. attributes	length	getNamedItem(*name*), setNamedItem(*name*) item(*index*), removeNamedItem(*name*)
DOMTokenList	Lista de clases de una etiqueta.	. classList	length value	add(), contains(), entries(), forEach(), item(*index*), keys(), remove(), replace(), values()

- **Diferencias** entre HTMLCollection y NodeList:
  - Se puede acceder a los elementos de **HTMLCollection** por su **nombre**, **id** o **índice**, pero a los de **NodeList** sólo se puede acceder por su **índice**.
  - Sólo el objeto **NodeList** puede contener **nodos de atributo** y **nodos de texto**.
  - **HTMLCollection** es **viva** y **NodeList** es **estática**, es decir, el primero reconocerá automáticamente un nuevo elemento añadido al DOM mientras que el otro no.

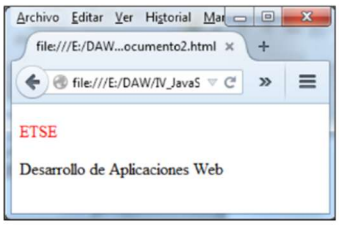
- **Similitudes** entre HTMLCollection y NodeList:
  - Son listas tipo **array** de **objetos**.
  - Tienen la propiedad . **length**.
  - Permiten acceso por **índices**.

MÉTODOS DE ACCESO A ELEMENTOS Y CONJUNTOS DE ELEMENTOS

- document. getElementById("*id*") → devuelve el **elemento** cuyo atributo **id** coincida con el parámetro.
- document. getElementsByClassName("*nombre*")→ devuelve una **HTMLCollection** con los elementos de la página con alguna **clase** igual al parámetro
- document. getElementsByName("*nombre*") → devuelve una **HTMLCollection** con los elementos de la página con atributo **name** igual al parámetro.
- document. getElementsByTagName("*HTML_tag*")→ devuelve una **HTMLCollection** con los elementos de la página con **etiqueta** igual al parámetro.
- document. querySelector("*CSS_Selector*") → devuelve el **primer elemento** de la página cuya etiqueta que coincida con el **selector CSS**.
- document. querySelectorAll("*CSS_Selector*") → devuelve un **NodeList** con todos los elementos de la página cuya etiqueta coincida con el **selector CSS**.

```html
<!DOCTYPE html>
<html>
    <body>
        <p class="daw">ETSE</p>
        <p class="daw">Desarrollo de Aplicaciones Web</p>

        <script>
            document.querySelector(".daw").style.color= "red";
        </script>
    </body>
</html>
```




```html
<ul class="miLista">
  <li>USC</li>
  <li>ETSE</li>
  <li>DAW</li>
  <li>POO</li>
  <li>IA</li>
</ul>

<p class="otraLista0"></p>
<p class="otraLista1"></p>
<p class="otraLista2"></p>
<ul class="otraLista3"></ul>
<ul class="otraLista4"></ul>
```

- USC
- ETSE
- DAW
- POO
- IA

Tipo de Objeto → [object NodeList]

Número de elementos → 3

Primer elemento de la colección → [object HTMLUListElement]

Contenido en texto del primer elemento → USC ETSE DAW POO IA

- USC
- ETSE
- DAW
- POO
- IA

Contenido del primer elemento

```
const miLista= document.querySelectorAll('ul');

document.querySelector(".otraLista0").innerHTML= miLista;
document.querySelector(".otraLista1").innerHTML= miLista.length
document.querySelector(".otraLista2").innerHTML= miLista[0]
document.querySelector(".otraLista3").innerHTML= miLista[0].textContent
document.querySelector(".otraLista4").innerHTML= miLista[0].innerHTML
```

```html
<ul class="miLista">
  <li>USC</li>
  <li>ETSE</li>
  <li>DAW</li>
  <li>POO</li>
  <li>IA</li>
</ul>

<p class="otraLista0"></p>
<p class="otraLista1"></p>
<p class="otraLista2"></p>
<ul class="otraLista3"></ul>
<ul class="otraLista4"></ul>
```

- USC
- ETSE
- DAW
- POO
- IA

Tipo de Objeto → [object HTMLCollection]

Número de elementos → 3

Primer elemento de la colección → [object HTMLUListElement]

Contenido en texto del primer elemento → USC ETSE DAW POO IA

- USC
- ETSE
- DAW
- POO
- IA

Contenido del primer elemento

```
const miLista_1= document.getElementsByTagName('ul');

document.querySelector(".otraLista0").innerHTML= miLista_1;
document.querySelector(".otraLista1").innerHTML= miLista_1.length;
document.querySelector(".otraLista2").innerHTML= miLista_1[0];
document.querySelector(".otraLista3").innerHTML= miLista_1.item(0).textContent;
document.querySelector(".otraLista4").innerHTML= miLista_1[0].innerHTML;
```

HTMLCollection vs NodeList → "Live" vs Estático

```html
<h3>Mi Lista</h3>
<ul id="miLista">
  <li class="item">USC</li>
  <li class="item">ETSE</li>
  <li class="item">DAW</li>
</ul>

<h3>Tamaño Antes</h3>
<p id="nListAntes"></p>
<p id="nListUlAntes"></p>
<p id="nColAntes"></p>
<p id="nColUlAntes"></p>

<h3>Tamaño Después</h3>
<p id="nListDespues"></p>
<p id="nListUlDespues"></p>
<p id="nColDespues"></p>
<p id="nColUlDespues"></p>
```

```
function showItems(items){

    let txt= "";
    for (let i=0; i<items.length-1; i++){
        txt+= items.item(i).textContent + ", ";
    }
    txt+= items.item(items.length-1).textContent;

    return(txt);
}
```

```
const miLista = document.getElementById('miLista');

const itemsByQuerySelector = document.querySelectorAll('.item');
const itemsByClassName = document.getElementsByClassName('item');

document.getElementById("nListAntes").innerHTML= 'NodeList: ' + itemsByQuerySelector.leng
document.getElementById("nListUlAntes").innerHTML= showItems(itemsByQuerySelector);
document.getElementById("nColAntes").innerHTML= 'HTMLCollection: ' + itemsByClassName.len
document.getElementById("nColUlAntes").innerHTML= showItems(itemsByClassName);

miLista.innerHTML += '<li class="item">POO</li>';

document.getElementById("nListDespues").innerHTML= 'NodeList: '+ itemsByQuerySelector.length;
document.getElementById("nListUlDespues").textContent= showItems(itemsByQuerySelector);
document.getElementById("nColDespues").innerHTML= 'HTMLCollection: ' + itemsByClassName.length;
document.getElementById("nColUlDespues").innerHTML= showItems(itemsByClassName);
```

itemsByQuerySelector → USC, ETSE, DAW

itemsByClassName → USC, ETSE, DAW

Mi Lista

- USC
- ETSE
- DAW
- POO

Tamaño Antes

NodeList: 3

USC, ETSE, DAW

HTMLCollection: 3

USC, ETSE, DAW

Tamaño Después

NodeList: 3

USC, ETSE, DAW

HTMLCollection: 4

USC, ETSE, DAW, POO

PROPIEDADES PARA ACCEDER AL CONTENIDO DE UN NODO

- *element*. nodeValue → devuelve el **contenido** del nodo actual.
- *element*. nodeName → devuelve el **nombre** del nodo actual.
- *element*. nodeType → devuelve el **tipo** del nodo actual.
- *element*. innerHTML → devuelve el **contenido** del nodo actual **interpretando** el html.
- *element*. textContent → devuelve el **contenido** del nodo actual (y de sus hijos) **sin interpretar** el html.

Tipos de nodo:
1 → Nodo elemento.
2 → Nodo atributo.
3 → Nodo texto.
8 → Nodo comentario.

```html
<ul id="miLista">
  <li>USC</li>
  <li>ETSE</li>
  <li>DAW</li>
</ul>

<p id="otroNodo"></p>
```

```
const x =document.getElementById("miLista");

let text = "";
text += "Name: " + x.nodeName + " - ";
text += "Value: " + x.nodeValue + " - ";
text += "Type: " + x.nodeType;
document.getElementById("otroNodo").innerHTML = text;
```

- USC
- ETSE
- DAW

Name: UL - Value: null - Type: 1

nodeName → UL; nodeValue → null; nodeType → 1

```html
<ul id="miLista">
  <li>USC</li>
  <li>ETSE</li>
  <li>DAW</li>
</ul>

<h3 id="Nodo">Mi Lista</h3>
<ol id="Lista"></ol>
<ol id="Lista1"></ol>
```

- USC
- ETSE
- DAW

Mi Lista

⇨

- USC
- ETSE
- DAW

USC

1. USC
2. ETSE
3. DAW

USC ETSE DAW

txt.firstElementChild.firstChild.nodeValue → USC

txt.innerHTML → 1. USC 2. ETSE 3. DAW

txt.textContent → USC ETSE DAW

```
const txt = document.getElementById("miLista");

document.getElementById("Lista").innerHTML= txt.innerHTML;
document.getElementById("Lista1").innerHTML= txt.textContent;
document.getElementById("Nodo").innerHTML= txt.firstElementChild.firstChild.nodeValue;
```

PROPIEDADES Y MÉTODOS PARA ACCEDER A ATRIBUTOS

- *element*.*atributo* → permite acceder al valor del atributo.
- *element*.getAttribute("*atributo*") → devuelve un **string** con el **valor** del atributo.
- *element*.getAttributeNode("*atributo*") → devuelve el **nodo del atributo** pasado como parámetro en forma de objeto **Attr**.
- *element*.attributes → devuelve los **atributos** de un elemento en forma de **NamedNodeMap** (compuesta por objetos Attr).
- *element*.getAttributeNames() → devuelve un **array** con los **nombres de los atributos** del elemento.

```
<h1>Atributos DOM</h1>
<img id="myImg" alt="logoUSC" src="logo.png" width="37" height="25">
<p>Los atributos de la imagen son:</p>
<p id="uno"></p>
<p id="dos"></p>
<p id="tres"></p>
```

Atributos DOM

USC

Los atributos de la imagen son:

atr → id= myImg alt= logoUSC src= logo.png width= 37 height= 25

.getAttribute("src") → logo.png

.getAttributeNode("alt").value → logoUSC

```
<script>
const nodeMap = document.getElementById("myImg").attributes;
let atr= "";
for (let i = 0; i < nodeMap.length; i++) {
  atr+= nodeMap[i].name + "= " + nodeMap[i].value + " ";
}
document.getElementById("uno").innerHTML= atr;
document.getElementById("dos").innerHTML= myImg.getAttribute("src");
document.getElementById("tres").innerHTML= myImg.getAttributeNode("alt").value;
</script>
```

MÉTODOS PARA MANIPULAR ATRIBUTOS

- *element*.setAttribute("*atributo*","*valor*") → **crea o modifica** un atributo.
- *element*.hasAttribute("*atributo*") → **comprueba si existe** el atributo.
- *element*.removeAttribute("*atributo*") → **elimina el valor** del atributo.

```
<script>
        var enlace = document.getElementById("miEnlace");
        enlace.setAttribute("href", "https://www.usc.es");
</script>
```

MÉTODOS PARA MANIPULAR LA ESTRUCTURA DEL ÁRBOL

- document.createElement("*HTML_tag*") → inserta un **nuevo nodo** en el DOM.
- document.createTextNode("*data*") → inserta un **nuevo nodo de texto** en el DOM.
- document.appendChild(*nodo*)/removeChild(*nodo*) → **inserta/elimina** un **nodo hijo** en el DOM.
- document.replaceChild(*nodonuevo*, *nodoviejo*) → **reemplaza** un **nodo hijo** en el DOM.

```
<h2 id="titulo"></h2>
```

```
var e= document.createElement("p");
var t= document.createTextNode("Universidad de Santiago de Compostela");
e.appendChild(t);
document.body.appendChild(e);

e1=document.getElementById("titulo")
e1.innerHTML = "Desarrollo de Aplicaciones Web";
```

Desarrollo de Aplicaciones Web

Universidad de Santiago de Compostela

PROPIEDADES PARA REFERENCIAR NODOS

- *element*.parentNode → selecciona el **nodo padre** del nodo actual.
- *element*.previousSibiling/nextSibling → selecciona el **nodo previo/siguiente** al nodo actual.
- *element*.firstChild/lastChild → selecciona el **primer/último hijo** del nodo actual.
- *element*.firstElementChild → selecciona el **primer elemento hijo** del nodo actual.
- *element*.childNodes → retorna los **nodos hijos** del nodo actual.
- *element*.children → retorna los **elementos hijos** del nodo actual.

```
<h1>Prueba Nodos</h1>
<p id= "titulo">Tipo Nodo</p><!-- Prueba Nodos -->

<p id="nodo_1"></p>
<p id="nodo_2"></p>
<p id="nodo_3"></p>
<p id="nodo_4"></p>
```

Prueba Nodos

Tipo Nodo

<p> con id= "titulo"

Name: P - Value: null - Type: 1

firstChild

Name: #text - Value: Tipo Nodo - Type: 3

nextSibling

Name: #comment - Value: Prueba Nodos - Type: 8

Attributes[0]

Name: id - Value: titulo - Type: 2

```
const x= document.getElementById("titulo");
const y= x.firstChild;
const z= x.nextSibling;
const a= x.attributes[0];

let txt_1 = "";
txt_1 += "Name: " + x.nodeName + " - ";
txt_1 += "Value: " + x.nodeValue + " - ";
txt_1 += "Type: " + x.nodeType;
document.getElementById("nodo_1").innerHTML = txt_1;

let txt_2 = "";
txt_2 += "Name: " + y.nodeName + " - ";
txt_2 += "Value: " + y.nodeValue + " - ";
txt_2 += "Type: " + y.nodeType;
document.getElementById("nodo_2").innerHTML = txt_2;

let txt_3 = "";
txt_3 += "Name: " + z.nodeName + " - ";
txt_3 += "Value: " + z.nodeValue + " - ";
txt_3 += "Type: " + z.nodeType;
document.getElementById("nodo_3").innerHTML = txt_3;

let txt_4 = "";
txt_4 += "Name: " + a.nodeName + " - ";
txt_4 += "Value: " + a.nodeValue + " - ";
txt_4 += "Type: " + a.nodeType;
document.getElementById("nodo_4").innerHTML = txt_4;
```

```
<body>
    <h1>Prueba Nodos</h1>
    <p id= "titulo">Tipo Nodo 1</p>
    <p>Tipo Nodo 2</p><p>Tipo Nodo 3</p>
    <!-- Prueba Nodos -->
    <p id="nodo"></p>
    <script> ... <script>
</body>
```

```
const nodeList = document.body.childNodes;

let text = "";
for (let i = 0; i < nodeList.length; i++) {
  text += nodeList[i].nodeName + "<br>";
}
document.getElementById("nodo").innerHTML = text;
```

Prueba Nodos

Tipo Nodo 1

Tipo Nodo 2

Tipo Nodo 3

#text
H1
#text
P
#text
P
P
#text
#comment
#text
P
#text
SCRIPT

PROPIEDADES Y MÉTODOS PARA ACCEDER A LOS ESTILOS

- $element.$style → permite acceder al **estilo de línea** del elemento seleccionado.
- window.getComputedStyle($elemento$, " ") → permite acceder a **todos los estilos relativos** al elemento seleccionado.

```
<h2 id="h_uno" style="color:white;">Estilos "inline"</h2>
<ul id="ul_uno"></ul>

<h2 id="h_dos">Estilos "outline"</h2>
<ul id="ul_dos"></ul>
```

```
<style>
  #h_dos {
    color: orange;
    background-color: black;
  }
</style>
```

```
document.getElementById("h_uno").style.backgroundColor= "black";

const estilo_h_uno= document.getElementById("h_uno").style;
const estilo_h_dos= document.getElementById("h_dos").style;

txt_1= showStyles(estilo_h_uno);
document.getElementById("ul_uno").innerHTML= txt_1;

txt_2= showStyles(estilo_h_dos);
document.getElementById("ul_dos").innerHTML= txt_2;
```

```
function showStyles(cssStyle){

    let txt= "";
    for (let i=0; i<cssStyle.length; i++){
        estilo= cssStyle.item(i);
        valor= cssStyle.getPropertyValue(estilo);
        txt+= "<li>" + estilo + ": " + valor + "</li>";
    }

    return(txt);
}
```

Estilos "inline"

- color: white
- background-color: black

Estilos "outline"

```
document.getElementById("h_uno").style.backgroundColor= "black";

const el_uno= document.getElementById("h_uno");
const el_dos= document.getElementById("h_dos");

const estilo_h_uno= window.getComputedStyle(el_uno);
const estilo_h_dos= window.getComputedStyle(el_dos);

txt_1= "<ul>";
txt_1+= showStyle(estilo_h_uno, "color");
txt_1+= showStyle(estilo_h_uno, "background-color");
txt_1+= "</ul>";
document.getElementById("ul_uno").innerHTML= txt_1;

txt_2= "<ul>";
txt_2+= showStyle(estilo_h_dos, "color");
txt_2+= showStyle(estilo_h_dos, "background-color");
txt_2+= "</ul>";
document.getElementById("ul_dos").innerHTML= txt_2;
```

ACCESO A EVENTOS

- Los EVENTOS ocurren cuando los usuarios realizan algún tipo de **acción** con el **teclado o el ratón** (click, cambiar el tamaño de la ventana, cargar la página, etc.). Cuando un evento ocurre, se **lanza** una **función** en concreto, que es **programable**.
- Hay 3 formas de ligar un evento a un elemento:
  - Mediante **atributos de eventos** → habitual en código antiguo, pero no recomendado (se mezcla JS con html).
    <elemento on$evento$ = "$funcion$()">
  - Mediante **manejadores de eventos DOM** → sólo permiten asociar una función por evento.
    document.getElementById().on$evento$ = $funcion$
  - Mediante **listeners de eventos DOM** → permiten asociar varias funciones por evento.
    document.getElementById().addEventListener("$event$", $funcion$);

```
<h1 id="cabecera"> USC </h1>
<p onclick="changeColor()">Pulsa para cambiar color a la cabecera</p>
```

```
function changeColor() {
  document.getElementById("cabecera").style.color= "red";
}
```

```
<h1 id="cabecera"> USC </h1>
<p id="pulsar">Pulsa para cambiar color a la cabecera</p>
```

```
document.getElementById("pulsar").onclick= changeColor;

function changeColor() {
  document.getElementById("cabecera").style.color= "red";
}
```

```
<h1 id="cabecera"> USC </h1>
<p id="pulsar">Pulsa para cambiar color a la cabecera</p>
```

```
document.getElementById("pulsar").addEventListener("click", changeColor);
document.getElementById("pulsar").addEventListener("click", changeText);

function changeColor() {
  document.getElementById("cabecera").style.color= "red";
}

function changeText() {
  document.getElementById("cabecera").style.textDecoration= "underline";
}
```

USC

Pulsa para cambiar color a la cabecera

USC

Pulsa para cambiar color a la cabecera

- **Flujo de eventos** → cuando un evento ocurre en una etiqueta interior a otras, el evento se propaga hacia las etiquetas que la contienen (false) o viceversa (true).

```
<h1 id="cabecera"> USC </h1>
<p id="pulsar">Pulsa para cambiar color a la cabecera</p>
<div id='out'>
   Caja Externa
       <div id='in'>
           Caja Interna
       </div>
</div>
```

```
#out{
    border-style:solid;
    display:inline-block;
    padding:10px
}
#in{
    border-style:dotted;
    display:inline-block;
    padding:10px
}
```

```
document.getElementById("out").addEventListener('click', changeRed, false);
document.getElementById("in").addEventListener('click', changeBlue, false);

function changeRed() {
  document.getElementById("cabecera").style.color= "red";
}

function changeBlue() {
  document.getElementById("cabecera").style.color= "blue";
}
```

USC

Pulsa para cambiar color a la cabecera

Caja Externa | Caja Interna

USC *False → Bubble Mode*

Pulsa para cambiar color a la cabecera

Caja Externa | Caja Interna

USC *True → Capturing Mode*

Pulsa para cambiar color a la cabecera

Caja Externa | Caja Interna

APIS JS

- Un API (Application Programming Interface) es un mecanismo que permite a un programa **interactuar con otros programas y/o scripts** mediante el **acceso a sus datos**.
- **JS** tiene muchas APIs **construidas sobre su núcleo** (no son parte del lenguaje) proporcionando nuevas **funcionalidades adicionales**.Se dividen en dos **categorías**:
  - APIs **de navegador** → integradas en el navegador, permiten **exponer datos del navegador** y del **entorno** informático circundante y trabajar con ellos.
    - ▷ Acceso al documento (API DOM), Audio/Vídeo (Web Audio, WebRTC), Manipulación de gráficos (Canvas, WebGL), Geolocalización, Storage, …
  - APIs **de terceros** → no están incluidas por defecto en el navegador y por ello es necesario **obtener** el **código** e **información** desde algún lugar de la **web**.

- API geolocation → permite averiguar la **posición del usuario** en términos de **coordenadas geográficas** de **latitud** y **longitud** siempre que éste proporcione el **permiso**.
  - Se implementa con el objeto geolocation, que es una propiedad del objeto navigator.

```html
<body>
   <h2>Geolocalización</h2>
   <p id="geoLoc"></p>
</body>
```

Geolocalización

Posición actual: (42.8834816, -8.5426176)

```html
<script>
function fnc(pos) {

  let x= "Posición actual: " +
  "(" + pos.coords.latitude +
  ", " + pos.coords.longitude + ")";

  document.getElementById("geoLoc").innerHTML= x;
}
navigator.geolocation.getCurrentPosition(fnc);
</script>
```

- API localStorage y sessionStorage → permiten el **almacenamiento local** de datos que pueden **compartirse** por las **diferentes páginas** del sitio web en el lado cliente.
  - ↳ Es un mecanismo alternativo a las cookies.
  - localStorage → los datos se almacenan **sin fecha de expiración**.
  - sessionStorage → los datos se almacenan **para la sesión** y se **pierden** al **cerrar la ventana** del navegador.
  - Hay dos formas de **almacenar los datos**:
    - Con el **método** → setItem(nombreDato, valorDato).
    - Usando el dato como **atributo** y dándole un valor → localStorage. nombreDato = valorDato;
  - Hay dos formas de **recuperar los datos**:
    - Con el método → valorDato = getItem(nombreDato)
    - Usando el dato como **atributo** y devolviendo el valor → valorDato = localStorage. nombreDato;

```html
<body>
    <h3 onclick="nClicks()">Pulsar</h3>
    <h4 id="resultado"></h4>
</body>
```

Pulsar

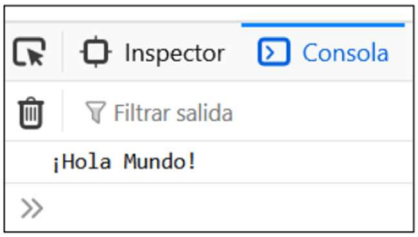
Pulsar

Has pulsado 6 veces.

```html
<script>
function nClicks() {
    if(typeof(Storage) !== "undefined") {
        if (localStorage.n)
            localStorage.n= Number(localStorage.n) + 1;
        else
            localStorage.n= 1;
        document.getElementById("resultado").innerHTML = "Has pulsaado " + localStorage.n + " veces.";
    }
}
</script>
```

- API history → permite **acceso al historial** de páginas visitadas por el navegador.
- API console → ofrece funcionalidad para realizar **tareas de depuración**, como **registrar mensajes** o los **valores de las variables** en puntos determinados del código, o **cronometrar** el tiempo que tarda en completarse una operación.
  - Se accede a través del objeto console que contiene muchos métodos con los que poder realizar tareas rudimentarias de depuración, generalmente centradas en el registro de varios colores en la Consola Web del navegador.
  - El método más usado es console. log (), que se usa para registrar el valor actual contenido dentro de una variable específica.

```html
<script>
  console.log("¡Hola Mundo!");
</script>
```



SINTAXIS BÁSICA

VARIABLES

- Las variables se pueden declarar de 3 formas:

	Ámbito de **bloque**	Permite **redeclaración**	Permite **reasignación**
var $nombre$; (en desuso)	No	Sí	Sí
let $nombre$;	Sí	No	Sí
const $nombre = valor$;	Sí	No	No

TIPOS DE DATOS

- JS es sensible al uso de **mayúsculas** y **minúsculas**.
- Los **comentarios** se escriben como en C.

- Cadenas de **texto** (string).
- Valores **numéricos** (number).
- **Booleanos** (boolean).
- **Objetos** (Object).
- **Nulo** (null).
- **No definido** (undefined).

- **typeof**$(variable)$ → devuelve el tipo de la variable.
- **Conversión** entre tipos:
  - **Implícita** → la coerción es automática hacia cadenas.
  - **Explícita** → mediante funciones como parseFloat(), parseInt(), toString(), String(), Number(), etc.

OPERADORES

- **Aritméticos** → − + ∗ / ++ −− %
- **Lógicos** → AND, OR, NOT
- **Relacionales** → == != > >= < <= === !==.
- **Bit** a bit → & | ^ ~ << >> >>>
- **Asignación** → = += −= ∗= /= |= ^= <<= >>= >>>=.
- **Otros** → + (concatenación de strings), . ?.
- El operador ... (spread) permite expandir los elementos de un objeto iterable.

```
const x = [3,5,1,7,5];  let mV = Math.max(...x);
```

- Las **operaciones** matemáticas generalmente se realizan por medio del **objeto Math**.
  - ↳ Usando with(Math) podemos predeterminar el objeto para no tener que escribirlo en cada ocasión.
  - **Métodos** incluidos → log(), exp(), sqrt(), pow(), abs(), floor(), ceil(), round(), randon(), sin(), cos(), tan(), max(), min(), etc.
  - **Constantes** incluidas → E, LN2, LN10, LOG2 ($\log_2(e)$), LOG10 ($\log_{10}(e)$), PI, SQRT2, SQRT1_2 ($1/\sqrt{2}$).

ESTRUCTURAS DE CONTROL

- **if** $(condicion)$ {...} else {...}
- **switch**$(expresion)$ {case $valor_1$: ... . break; case $valor_2$: ... break; ...}
- **for**$(valor\ inicial; condición\ parada; incremento)$ {...}
- **for** $(dato$ **in** $Objeto.valor)$ {...} → itera sobre las claves (en un array, los índices).
- **for** $(dato$ **of** $Objeto.valor)$ {...} → itera sobre los valores-
- **while**$(condicion)$ {...}
- **do** { ... } **while**$(condicion)$

FUNCIONES

```
function miFuncion(...miPar)
  let k= 0;
  for (let k of miPar)
      s+= k
}

miFuncion(1,2,3,3,3)  → 12
```

- Las funciones se definen de la siguiente forma: **function** $nombre(parametros)$ {... **return**(); }.
  - **No** se especifica el **tipo** de los **parámetros** ni del **retorno**.
  - Se pueden establecer **valores por defecto** para los parámetros.
  - Se permite establecer un **número indefinido** de **parámetros** usando el operador ...
  - Los **argumentos** de tipo **primitivo** se pasan por **valor** y los **objetos** por **referencia**.
- De manera alternativa, se pueden definir como **funciones anónimas** almacenadas en una variable $x =$ **function**$(parametros)$ {... **return**()}.
  - Esto se puede hacer de manera resumida de la siguiente forma: $x = (parametros) =>$ {...};.
    - ↳ Si la función tiene exactamente un argumento se pueden omitir los ().

- Las funciones pueden **autoinvocarse**, en cuyo caso no llevan return: (function$(parametros)$ {...})$(valor_parametros)$;.
- Una función puede ser **pasada como argumento** de otra y ser ejecutada por esta última.
- Las funciones son **objetos** en JS, por lo que tienen propiedades y métodos:
  - arguments → array que almacena los argumentos.
  - arguments.length → número de argumentos.
  - toString() → retorna la función como cadena.

```
function miF(a){
  return a + 100;
}
```

⇨

```
miF= a => a + 100;
```

```
(function(a, b) {
  c=0;
  if (a>0)
    c= a+b;
  else
    c=a-b;
  document.getElementById("uno").innerHTML = c
})(3, 4);
```

Funciones JavaScript

Resultado de la llamada a la función: 7

```
x= function myFunction(a, b) {
  c=0;
  if (a>0)
    c= a+b;
  else
    c=a-b;
  return c;
}
document.getElementById("uno").innerHTML = x(3, 4);
```

Funciones JavaScript

Resultado de la llamada a la función: 7

```
function miVisualizacion(dato) {
  document.getElementsByTagName("p")[0].innerHTML= dato;
}

function miCalculo(x1, x2, miFuncionCallback) {
  let resultado= x1 * x2;
  miFuncionCallback(resultado);
}

miCalculo(5, 4, miVisualizacion);
```

OBJETOS

INSTANCIACIÓN

- Tradicionalmente, la instanciación de objetos se realizaba mediante new, que invoca la función constructor.
  - ↳ Estos constructores podían llamar al constructor de la clase padre con super().
- La tendencia actual es usar Objeto nombre_objeto $= \{atr_1: valor_1, \dots, atr_n: valor_n,\ funcion(){\dots}, \dots\}$ para instanciarlo directamente.

PROPIEDADES

- Los atributos de la clase se denominan **propiedades**, y se puede acceder a ellas usando o bien $objeto.atributo$ o bien $objeto["atributo"]$.
- Las propiedades de un objeto son **públicas** y se pueden modificar directamente $objeto.atributo = valor$.

MÉTODOS

- La llamada a los métodos de un objeto se realiza usando $objeto.funcion(parametros)$.

ARRAYS

- Los arrays son **objetos**, por lo que se pueden inicializar con new, pero se recomienda usar let $array = [\ ]$.
  - Tienen **tamaño dinámico**.
  - Son **heterogéneos**, es decir, pueden contener elementos de distintos tipos.
- Propiedades y métodos → length(), join(), reverse(), sort(), concat(), slice().
  - from() → retorna un array desde un objeto iterable.
  - keys() → retorna un iterador con los índices del array.
  - find() → retorna el primer elemento del array que verifica una función de testeo.
  - findIndex() → retorna el índice del primer elemento del array del array que verifica una función de testeo.

Existen diversas formas para **iterar** un array:

- forEach($funcion$) → llama a funcion una vez para cada elemento.
- map($funcion$) → crea un nuevo array aplicando funcion una vez para cada elemento.
- filter($funcion$) → crea un nuevo array aquellos los elementos del original que devuelven true para $funcion$.
- every(funcion) → devuelve true si todos los elementos del array devuelven true para la $funcion$ y false en caso contrario.
- some($funcion$) → devuelve true si algún elemento del array devuelve true para la $funcion$ y false en caso contrario
- ↳ La $funcion$ toma los argumentos (value, index, array).
- reduce($funcion$, [$valor_inicial$]) → aplica la $funcion$ a cada elemento del array de izquierda a derecha, produciendo un único valor como resultado.
- reduceRight($funcion$, [$valor_inicial$]) → aplica la $funcion$ a cada elemento del array de izquierda a derecha, produciendo un único valor como resultado.
- ↳ La $funcion$ toma los mismos argumentos que la anterior, añadiendo total (valor inicial/valor anterior).

CADENAS

- Las cadenas son **objetos**, por lo que se pueden inicializar con new, pero se recomienda usar let $cadena = "\dots"$.
- Métodos → fromCharCode(), charCodeAt(), indexOf(), lastIndexOf(), substr(), slice(), substring(), toLowerCase(), toUpperCase() includes(), startsWith(), endWith().

PROGRAMACIÓN ORIENTADA A EVENTOS

- click → pulsar el ratón.
- mouseover → posicionarse encima con el ratón.
- mouseout → dejar de posicionarse encima con el ratón.
- load → cargar página.
- unload → abandonar página.
- focus → ventana pasa a ser activa.
- blur → ventana deja de ser activa.
- keypress → presionar tecla.
- select → seleccionar.
- change → modificar.
- submit → enviar formulario.
- reset → reiniciar formulario.
- error → se produce un error.
- abort → se aborta.

window	load, unload, blur, focus
link	click, mouseout, mouseover
area	mouseOut, mouseOver
image	abort, error, load
form	reset, submit
text, textarea, password	blur, change, focus, select
button, reset, submit, radio, checkbox	click
select	blur, change, focus
Controles de selección de archivos	blur, focus, select

FECHA Y HORA

- La fecha y la hora se tratan con el objeto **date**.Tiene dos constructores.
  - new Date("$mes, día, año, horas: minutos: segundos$").
  - new Date($año, mes, día$, [$horas$, [$minutos$, [$segundos$, [$milisegundos$]]]]).
  - getTime(), getDay(), getDate(), getMonth(), getYear(), getFullYear(), getHours(), getMinutes(), getSeconds(), getMiliseconds(), setTime(), setDay(), setDate(), setMonth(), setYear(), setFullYear(), setHours(), setMinutes(), setSeconds(), setMiliseconds().
- También se pude usar la Hora Universal Coordinada (UTC)/Hora GMT (Greenwich Meridian Time).
  - Tiene métodos análogos a los anteriores con UTC después del get/set.
  - Conversiones → toUTCString(), toGMTString().

JAVASCRIPT ES6

- En 2015 ECMAScript hizo una **revisión importante del JS**, creándose la versión denominada ES6 o ECMAScript6.
  - ↳ Ahora las revisiones se siguen haciendo anualmente, pero con menor intensidad.

Introdujo:

▪ **let y const** para la declaración de variables.	▪ Los lazos **for/of**.	▪ Nuevos métodos para **String** → includes(), startsWith(), endWith().
▪ El operador ...	▪ Definición de **constructores sin** poner **function**.	▪ Nuevos métodos para **Array** → from(), keys(), find(), findIndex().
▪ **Valores por defecto** para funciones.	▪ **Módulos**.	▪ Nuevos métodos para **Math**.
▪ Funciones con **número indefinido de parámetros**.	▪ **Mapas y conjuntos**.	▪ Nuevos métodos y propiedades para **Number**.
▪ Funciones **resumidas** con =>.	▪ **Símbolos**.	▪ Nuevos métodos globales → **isFinite**() e **isNaN**()

MÓDULOS

- Los MÓDULOS son ficheros que incluyen **colecciones** de **funciones** y/o **clases** con un **propósito común**.
  - Suelen agruparse en estructuras de **carpetas separadas**, sin necesidad de **ningún nombre en especial**.
    - ↳ En los últimos años se ha planteado el uso de la extensión . mjs pero no está todavía generalizada.
  - ❖ Se invocan a través de **servidores**.
- Se pueden **importar** de dos formas distintas:
  - Importar nombres exportados desde JS → los nombres sólo se pueden cambiar con as (p. e.$\{miF \text{ as } mF\}$).

```
import {miF, nombre} from "./miJS.js"
```

Donde en miJS.js:

```
export const nombre= "Pepe"
export function miF()
```

  - Importar nombres exportados desde JS por defecto → el nombre se puede cambiar directamente
    - ↳ Se puede tener un default y no default en el mismo módulo

```
import miFNueva from "./miJS.js"
```

Donde en miJS.js:

```
export default function miF()
```

- Para **invocar los módulos en html** hay que incluir el atributo type = "module" en la etiqueta <script>.

```
index.html
main.js
modules/
  miModulo_1.js
  miModulo_2.js
```

```
<h3>Funciones JavaScript</h3>
<p>Resultado de la llamada: <b id="uno"></b></p>
```

Funciones JavaScript

Resultado de la llamada: 7

```
<script type="module">
    import {suma} from "./miModulo.js";
    let x = suma(4, 3);
    document.getElementById("uno").innerHTML = x;
</script>
```

```
export function suma(a, b) {
    return a+b;
}

function resta(a, b) {
    return a-b;
}
```

miModulo.js

MAPAS Y CONJUNTOS

- MAPAS → conjunto de datos creado por pares (clave, valor) a los que se accede por el valor de su clave.
  - Las **claves** pueden ser **objetos**.
  - Son **iterables** en **orden**.
  - ▪ new Map() → constructor.
  - ▪ set($clave, valor$) → fija un valor para un elemento del mapa.
  - ▪ get($clave$) → retorna un valor para un elemento del mapa.
  - ▪ clear() → borra todos los elementos del mapa.
  - ▪ keys() → retorna un objeto iterador con todas las claves del mapa.
  - ▪ values() → retorna un objeto iterador con todos los valores del mapa.
- SETS → colección de datos con valores únicos.
  - ▪ new Set() → constructor.
  - ▪ add($valor$) → añade un valor al conjunto.
  - ▪ delete($valor$) → elimina un elemento del conjunto.
  - ▪ clear() → borrar todos los elementos del conjunto.
  - ▪ values() → retorna un objeto iterador con todos los valores del conjunto.

SÍMBOLOS

- Symbol es un nuevo tipo de **dato primitivo** que representa un **identificador único oculto** al que **no** puede **acceder otro código** por accidente.
  let $nombre$ = Symbol($'nombre'$).
- Se puede acceder a ellos con [], pero no con '.'

```
class miClase {
   ......
}

let id= Symbol('id')
miClase[id]= 11111;        → miClase[id] es accesible, pero
                             miClase.id no
```

EXPRESIONES REGULARES

- Durante el proceso de **intercambio de datos** con el **servidor** se han de intercambiar **datos consistentes**, para **no corromper** los datos almacenados en las **bd** asociadas. Por tanto, los datos se analizan para comprobar que los **campos obligatorios están cubiertos** y que la **información enviada es del tipo correcto**.

○ Las EXPRESIONES REGULARES permiten definir el **patrón** de una **cadena** de caracteres y utilizarlo como **mecanismo de control** de la información.

Se pueden definir de dos formas:

- Mediante barras inclinadas → /patrón/opciones.
  ▷ let miER =/[A − Z/i
- Mediante el constructor de la clase RegExp → RegExp("patron", ["opciones"]).
  ▷ let miER = new RegExp("[A − Z], "i").

Opciones:
- i → búsqueda sin tener en cuenta mayúsculas.
- g → búsqueda global en toda la cadena.
- m → búsqueda en textos multilínea.

CLASE REGEXP

Métodos:
- exec() → comprueba coincidencias con una cadena de texto y retorna la primera.
- test() → comprueba coincidencias con una cadena de texto y retorna true o false.
- toString() → retorna el valor de la ER en formato de cadena (heredado de Object).

Propiedades:
- global → comprueba si el modificador g está activado.
- ignoreCase → comprueba si el modificador i está activado.
- multiline → comprueba si el modificador m está activado.
- lastIndex → especifica la posición en la que comienza la siguiente coincidencia.
- source → devuelve el valor de la ER en formato cadena.

Ejemplo: partes de una dirección de e-mail.

```
const text= "pablo.tahoces@usc.es"
re= (/([a-z0-9\-\.]+)@([a-z0-9\-\.]+\.[a-z]{2,})/i

let email= text.match(re);
```

Almacena:
- email[0].- dirección completa.
- email[1].- usuario.
- email[2].- dominio.

CLASE STRING

Se pueden usar métodos del objeto String para trabajar sobre ERs.
- match() → busca coincidencias usando una ER y devuelve un array con los resultados.
- matchAll() → busca coincidencias usando una ER y devuelve un iterador con los resultados.
- search() → busca coincidencias usando una ER y devuelve el índice donde encuentra la primera.
- replace() → reemplaza texto en una cadena usando una ER y devuelve la cadena modificada con todas las coincidencias, aunque no se use /g.
- split() → trocea una cadena en partes usando una ER como separador y devuelve un array con esas partes.

PATRONES

- **Cuantificadores** → agregan datos sobre cantidades de caracteres a las expresiones regulares.
  - ∗ → cero o más veces.
  - + → una o más veces.
  - ? → cero o una vez.
- **Caracteres de escape** → localizan caracteres no imprimibles y representan caracteres con significado especial en la expresión.
  - \ → carácter de escape.
- **Clases o conjuntos de caracteres** → sirven para localizar cualquiera de los caracteres que forman parte de un conjunto.
  - [*conjunto de caracteres*] → caracteres que pueden formar parte de la cadena.
    ▷ [A − Z a − z]
  - *conjunto de caracteres* \b → válido si una palabra de la cadena acaba en *conjunto de caracteres*.
    ▷ /[a − f]\b/g Hol**a** mundial
  - *conjunto de caracteres* \B → válido si una palabra de la cadena no acaba en *conjunto de caracteres* pero sí lo contiene.
    ▷ /[a − f]\B/g Hola mun**dia**l
- **Caracteres de marcado o posición** → marcan una posición específica dentro de la cadena.
  - ^ → comienzo de la cadena o línea.
    ▷ /^[A − Z]/g **H**ola Mundial
  - $ → final de la cadena o línea.
    ▷ /[0 − 9]$/g 1234**5**
- **Alternancias** → permiten definir opciones para una expresión.
  ▷ /[a | b]\b/g Hol**a** mundial
- **Agrupaciones** → permiten capturar subpatrones, dentro de un patrón.
  ▷ (A − Z a − z)
- **Subpatrones en cadenas** → permiten definir opciones para una expresión.
  - ?: → busca coincidencias con la expresión.
    ▷ /[a − z](? : [A − Z])/g Hol**a** Mundial
  - ? = → captura una coincidencia si la primera expresión está seguida de la segunda.
    ▷ /[a − z](? = [A − Z])/g Hol**a** Mundial
  - ? ! → captura una coincidencia si la primera expresión no está seguida de la segunda.
    ▷ /[a − z](? ! [A − Z])/g H**o**la M**undial**

Precedencia entre operadores:
1. \
2. () []
3. ∗ + {n} {n, } {n, m}
4. ^ $
5. |

- **Otros patrones**.
  - {n} → coincide exactamente n veces.
  - {n, } → coincide como mínimo n veces.
  - {n, m} → coincide n veces como mínimo y m veces como máximo.
  - \d → se sustituye por [0 − 9].
  - \w → se sustituye por [A − Z a − z 0 − 9 _]
  - \W → \w negado.

```
re= /([a-z0-9\-\.]+)@([a-z0-9\-\.]+\.[a-z]{2,})/i

let email= re.exec(SEMail);
```

Almacena:
- email[0].- dirección completa.
- email[1].- usuario.
- email[2].- dominio.

```
Function isEmail(sEMail)
{
    re= /[a-z0-9\-\.]+@[a-z0-9\-\.]+\.[a-z]{2,}/i

    if (re.test(SEMail)
        return true;
    else
        return false;
}
```

VALIDACIÓN MEDIANTE JS

```html
<form action="miPagina.html">
  <label for="fname">Introduce nombre usuario:</label>
  <input type="text" placeholder= "Nombre susario">
  <label for="fname">Contraseña:</label>
  <input type="text" placeholder= "Palabra de paso">
  <input type="submit" value="Enviar">
</form>
```

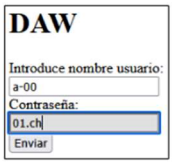
```js
const form= document.getElementsByTagName("form")[0];
form.addEventListener("submit", validarForm);
```

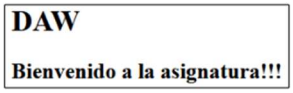
```js
function validarForm(e){
    e.preventDefault();
    let msg= "";

    campo= form[0];
    re= /[a-h]{1,3}\-[0-5]{2,3}/i;
    msg= "Introduce un nombre válido";
    if (!re.test(campo.value)){
        changeForm(msg, campo)
        return;
    }

    campo= form[1];
    re= /[0-3]{2,3}\.[ach]{1,2}/i
    msg= "Introduce un pwd válido";
    if (!re.test(campo.value)){
        changeForm(msg, campo)
        return;
    }

    form.submit();
}
```



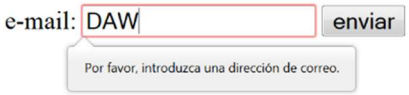



```js
function changeForm(msg, campo){
    alert(msg);
    campo.focus();
    campo.select();
}
```

VALIDACIÓN MEDIANTE HTML5

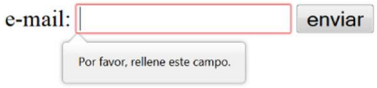
- HTML5 proporciona **nuevos atributos** para la etiqueta **<input>** que permiten realizar **validaciones** de elementos.
  - type = "email" → análogo a text pero para direcciones de email. En caso de introducir una dirección no válida solicita de nuevo un valor.
  - type = "url" → funcionamiento similar a email.
  - type = "number" → funcionamiento similar a email.
  - Atributo pattern → permite insertar un patrón en formato de ER.
  - Atributo required → indica que es obligatorio rellenar el elemento (es booleano).

```
e-mail: <input type="email" name="mail">
<input type="submit" value="enviar">
```




```
Ej.-
Numero: <input type="number" name="number" pattern="[0-9]{1,3}">
<input type="submit" value="enviar">
```

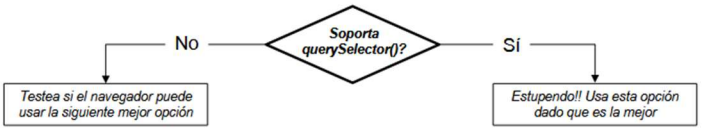
```
e-mail: <input type="email" name="mail" required>
<input type="submit" value="enviar">
```



JQUERY

- Fue creada por John Resig y presentada en enero de 2006 en el BarCamp de NYC.
- JQUERY es una librería de funciones escrita JS libre y de **código abierto** para la **simplificación** de la **interacción** con documentos **HTML**.
  - Manipulación del árbol **DOM**.
  - Manejo de **eventos**.
  - Desarrollo de **animaciones**.
  - Interacción mediante **AJAX** en páginas web.

```
<script src="jquery.js"></script>
<script src="http://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery
.min.js"">
</scrip
```



- Ventajas respecto al uso de métodos DOM
  - ✓ Permite **selecciones** de **etiquetas HTML** de una manera más **simple**.
  - ✓ Una vez hecha una **selección** es posible aplicarle **diferentes métodos**.
  - ✓ Es **robusto** frente a la acción de los **diferentes navegadores**.
  - ✓ El **manejo de eventos** es mucho más **sencillo** y funciona en la mayoría de los **navegadores**.
  - ✓ Existen métodos adicionales que permiten hacer **animaciones más sofisticadas**.
- jQuery gestiona las **inconsistencias** que ocurren con las diferentes formas en que los **navegadores seleccionan** y **manejan** los **eventos** analizando si el navegador puede usar la **forma ideal** y en caso contrario buscando el **procedimiento más adecuado**.

SELECCIÓN DE ELEMENTOS

- La sintaxis básica de jQuery es $(selector).action().
  - $ → forma abreviada de llamar al **constructor** de la clase jQuery.
  - selector → parámetro del constructor, sirve para **seleccionar con estilo CSS** qué etiquetas HTML se seleccionan para la acción.
  - Cuando se realiza una selección el objeto creado contiene un atributo llamado **length**, cuyo valor es el **número de etiquetas** de ese tipo **seleccionadas** y por tanto, accesibles desde el objeto.
  - ▷ $('ul li').addclass('etse').
- Método ready() → informa de cuando el **DOM** del documento html está **listo** y entonces **ejecuta la función** que lleva como argumento.
  $(document).ready(function(){ ... });
  - ↳ Una versión resumida es $(function(){... });

```
<h3>Escuela Técnica de Ingenieria</h3>
<p></p>
```

Escuela Técnica de Ingeniería

Desarrollo de Aplicaciones Web

```
$(function(){
    $("p").text("Desarrollo de Aplicaciones Web");
    $("h3").css('color', 'blue');
});
```

Selectores específicos jQuery.

$(this)	Selecciona la **etiqueta actual**.
$("p: first")	Selecciona la **primera** etiqueta **<p>** (ídem con p: last).
$("ul li: first-child")	Selecciona la **primera** etiqueta **<li> de cada** etiqueta **<ul>**.
$(": button")	Selecciona todas las **etiquetas <button>** y todas las **etiquetas <input> con type="button"**.
$("tr: even")	Selecciona todas las **etiquetas <tr> pares** (ídem con tr: odd).

MÉTODOS JQUERY

ACCESO/MODIFICACIÓN DE CONTENIDO

- Permiten acceder/modificar el **contenido** de una **etiqueta** html.
- html() → retorna/cambia el **contenido html**, incluyendo el texto (**interpreta** el html).
- text() → retorna/cambia el **contenido de texto** (**no interpreta** el html).
- replaceWith() → reemplaza el **elemento entero** por otro, sustituyendo el nodo entero (interpreta el html).
- remove() → **elimina** los nodos seleccionados.

```
<body>

  <h3>Escuela Técnica de Ingeniería</h3>

</body>
```

Archivo | F

Escuela Técnica de Ingeniería

```
<body>
   <h2>Escuela Técnica de Ingeniería</h2>
   <p>USC</p>
   <div id= "uno"></div>
   <div id= "dos"></div>
</body>
```

Universidad de A Coruña

Santiago de Compostela

<h1>Santiago de Compostela</h1>

```
<script>
  $(document).ready(function(){
     var txt= $("h3").text();
          $("h3").html("<i>" + txt + "</i>");
  });
</script>
```

```
<script>
   $(document).ready(function(){
      $("h2").remove();
      $("p").replaceWith("h3>Universidad de A Coruña</h3>");

      txt="<h1>Santiago de Compostela</h1>"
      $("#uno").html(txt)
      $("#dos").text(txt)
   });
</script>
```

ACCESO/MODIFICACIÓN DE NODOS

- Permiten acceder/modificar los **nodos** del árbol **DOM**.
- before() → inserta contenido **antes** de los **elementos** seleccionados.
- after() → inserta contenido **después** de los **elementos** seleccionados.
- prepend() → inserta contenido **después** de la **etiqueta de apertura** del elemento seleccionado.
- append() → inserta contenido **antes** de la **etiqueta de cierre** del elemento seleccionado.
- clone() → crea una **copia** de los elementos seleccionados, incluyendo **nodos de texto y descendientes**.
- unwrap() → **elimina padres** de un elemento seleccionado, pero **preservando** los **elementos** seleccionados.
- detach() → lo mismo que remove(), pero preservando una **copia** en memoria.
- empty() → elimina **nodos hijo y descendientes** de elemento seleccionado.
- add() → añade **nuevo contenido** a los elementos seleccionados.

```
<p>Ejemplo Append.</p>
<p>Otro Ejemplo Append.</p>
<p>Ejemplo Prepend.</p>
```

Ejemplo Append.

Otro Ejemplo Append.

Ejemplo Prepend.

```
$(document).ready(function(){
  $("p:first").click(function(){
    $("p:first").append("<b>Texto añadido.</b> ");
    $("p").eq(1).append("<b>Texto añadido.</b> ");
  });

  $("p:last").click(function(){
    $("p:last").prepend("<b>Texto añadido.</b>");
  });

});
```

Ejemplo Append.**Texto añadido.**

Otro Ejemplo Append.**Texto añadido.**

Texto añadido. Ejemplo Prepend.

ACCESO/MODIFICACIÓN DE ATRIBUTOS Y ESTILOS

- Permiten acceder/modificar los **atributos** de las etiquetas y/o el **estilo** aplicado.
- attr() → **retorna/cambia el atributo**, entre paréntesis, del elemento seleccionado.
- removeAttr() → **elimina el atributo**, entre paréntesis, del elemento seleccionado.
- addClass() → **añade un atributo class** con su valor a un elemento seleccionado.
- removeClass() → **elimina un atributo class** de un elemento seleccionado.
- css() → **retorna/cambia propiedades CSS** del elemento seleccionado.

```
<body>
   <h2>Escuela Técnica de Ingeniería</h2>
   <p><a href="http://www.usc.es">USC</a></p>
</body>
```

```
<script>
   $(document).ready(function(){
      $("a").removeAttr('href')
      $("p").css('color', 'green');
   });
</script>
```

ACCESO A FORMULARIOS

- Permiten acceder al **contenido** de las **etiquetas** de los **formularios**.
- val() → **retorna/fija el valor** de los elementos tipo <input>, <select>, <textarea>.
- isNumeric() → retorna true si el valor entre paréntesis es de tipo numérico.

```
<h2>Escuela Técnica de Ingeniería</h2>
<input type="text">
<p></p>
```

```
<script>
$(document).ready(function(){
  $("input").change(function(){
     $("p").text($("input").val());
  });
});
</script>
```

SELECCIÓN DE ELEMENTOS

- Permiten seleccionar los diferentes **elementos** del árbol **DOM**.
- find() → dentro de la actual selección, retorna todos los elementos con **etiqueta** que **coincida** con el valor del parámetro.
- closest() → retorna el **ancestro** del elemento seleccionado con **etiqueta** que **coincida** con el valor del parámetro.
- parent() → retorna el **padre** del elemento seleccionado.
- parents() → retorna **todos los padres** del elemento seleccionado.
- children() → retorna **todos los hijos** del elemento seleccionado.
- siblings() → retorna **todos los hermanos** del elemento seleccionado.
- next() → retorna el **siguiente hermano** del elemento seleccionado.
- nextAll() → retorna **todos los subsecuentes hermanos** del elemento seleccionado.
- prev() → retorna el **hermano anterior** al elemento seleccionado.
- prevAll() → retorna **todos los hermanos anteriores** al elemento seleccionado.

```
<h2>Escuela Técnica de Ingeniería</h2>
<input type="text">
<p></p>
```

```
<script>
$(document).ready(function(){
  $("input").change(function(){
     $("p").text($("input").val());
  });
});
</script>
```

LAZOS

- Permiten **recrear** la funcionalidad de los **lazos** en una selección de elementos.
- each() → ejecuta una o más instrucciones para cada uno de los elementos seleccionados.
  - ↳ Los ítems seleccionados son accesibles mediante el objeto this.

```
<body>
  <div>
    <h2>Lista de Titulaciones</h2>
    <ul>
      <li>Ingeniería Química</li>
      <li>Ingeniería Ambiental</li>
      <li>Ingeniería Informática</li>
    </ul>
  </div>
</body>
```

```
<script>
$(document).ready(function(){
  $("ul").parent().css("border", "solid");
  $("ul").siblings().css("color", "red");
  $("ul").children().css("color", "blue");
});
</script>
```

```
<h2>Escuela Técnica de Ingeniería</h2>
<p><a href="http://www.usc.es">USC</a></p>
<p>Santiago de Compostela</p>
```

```
<script>
  $(document).ready(function(){
     $("a").removeAttr('href')
     $("p").has("a").css('color', 'red');
  });
</script>
```

DE TESTEO Y FILTRADO

- Permiten realizar operaciones de **filtrado complejas**.
- filter() → filtra los elementos seleccionados con la **selección CSS pasada** como argumento.
- not() → filtra los elementos seleccionados con la **selección CSS** pasada como argumento, **negada**.
- has() → filtra los elementos seleccionados que tengan un **descendiente** que coincida con la **selección CSS** pasada como argumento.
- is() → retorna un **booleano** que indica si la selección de elementos cumple la **selección CSS** pasada como argumento.
- $.contains($nodopadre, nodohijo$) → comprueba si un **elemento está dentro de otro** y devuelve un **booleano**.

DE ORDEN EN SELECCIÓN

- Permiten realizar operaciones de **selección complejas**.
- eq() → selecciona de entre los elementos seleccionados, aquel cuyo **índice** es el parámetro.
- lt() → selecciona de entre los elementos seleccionados, aquellos cuyo **índice es menor** que el parámetro.
- gt() → selecciona de entre los elementos seleccionados, aquellos cuyo **índice es mayor** que el parámetro.
- ↳ Todos los elementos retornados por jQuery en una selección tienen un índice que puede ser usado como filtro.

```
<h2>Escuela Técnica de Ingeniería</h2>
<ul>
  <li>Ingeniería Quimica</li>
  <li>Ingeniería Ambiental</li>
  <li>Ingeniería Informática</li>
</ul>
```

```
<script>
  $(document).ready(function(){
     $("li").eq(2).css('color', 'red');
  });
</script>
```

DE DIMENSIÓN

- Permiten retornar o fijar **dimensiones de las cajas** de los elementos seleccionados.
- height() → retorna o fija la altura de la caja (**sin margen, borde o padding**).
- width() → retorna o fija la anchura de la caja (**sin margen, borde o padding**).
- innerHeight() → retorna o fija la altura de la caja (**sin margen o borde**).
- innerWidth() → retorna o fija la anchura de la caja (**sin margen o borde**).
- outerHeight() → retorna o fija la altura de la caja (**sin margen**).
- outerWidth() → retorna o fija la anchura de la caja (**sin margen**).
- $(document).height() → retorna la **altura del documento**.
- $(document).width() → retorna la **anchura del documento**.
- $(window).height() → retorna la **altura de la ventana** en donde se abre el documento.
- $(window).width() → retorna la **anchura de la ventana** en donde se abre el documento.

DE POSICIÓN

- Permiten retornar o fijar **posiciones de las cajas** de los elementos seleccionados.
- offset() → recupera o fija la posición de un elemento **relativa a la esquina superior izquierda del documento**.
- position() → recupera o fija la posición de un elemento **relativa a su ancestro**.
- scrollLeft() → retorna la posición del **scroll horizontal**, con respecto al lateral izquierdo del documento.
- scrollTop() → retorna la posición del **scroll vertical**, con respecto a la parte superior del documento.

```
<h1>Posiciones</h1>
<h2>Estas son las posiciones</h2>
<p>Posición respecto a top: </p>
<p>Posición respecto a left: </p>
```

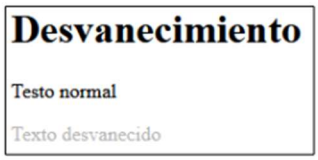
```
<script>
$(document).ready(function(){
    let pos = $("h2").offset();
    $("p:first").append(pos.top);
    $("p:last").append(pos.left);
});
</script>
```

DE ANIMACIÓN

- Permiten **añadir animaciones** a ciertas partes de la página.
- show() → **muestra** los elementos seleccionados.
- hide() → **oculta** los elementos seleccionados.
- toggle() → **muestra/oculta** los elementos seleccionados.
- fadeIn() → cambia la opacidad de los elementos seleccionados **hasta hacerlos opacos**.
- fadeOut() → cambia la opacidad de los elementos seleccionados **hasta hacerlos transparentes**.
- fadeTo() → cambia la opacidad de los elementos seleccionados.
- fadeToggle() → cambia la opacidad de los elementos seleccionados, **alternando sus valores**.
- slideDown() → **muestra** los elementos seleccionados, mediante efecto **deslizamiento**.
- slideUp() → **oculta** los elementos seleccionados, mediante efecto **deslizamiento**.
- slideToggle() → **muestra/oculta** los elementos seleccionados, mediante efecto **deslizamiento**.
- delay() → **retrasa** la **ejecución** del **siguiente ítem** en la cola.
- stop() → **detiene** una **animación** en curso.
- animate() → crea **animaciones personalizadas**.

```
<h1>Desvanecimiento</h1>
<p>Testo normal </p>
<p>Texto desvanecido</p>
```

```
$(document).ready(function(){
    $("p:last").fadeTo("slow", 0.3);
});
```



EVENTOS JQUERY

- jQuery es capaz de responder a los eventos típicos mediante funciones manejadoras de eventos, de dos formas
  - $\$(selector).evento(\mathbf{function}(e)\{\ldots\});$
  - $\$(selector).\mathrm{on}('evento', \mathbf{function}(e)\{\ldots\});$

- Las **funciones manejadoras** de eventos reciben un **objeto evento** como argumento que incorpora **propiedades y métodos propios del evento** que ocurre.
  - type → **tipo** de evento (click, etc.).
  - which → **quién** lo desencadena (botón de ratón, tecla, etc.).
  - target.nodeName → **nombre** del elemento seleccionado.
  - pageX → **posición del ratón**, referenciada al lado izquierdo del documento.
  - pageY → **posición del ratón**, referenciada al lado derecho del documento.
  - preventDefault() → **evita** que ocurra lo que tiene que ocurrir por **defecto**.
  - stopPropagation() → **detiene la propagación** del evento hacia sus ancestros.

Tipos de evento:

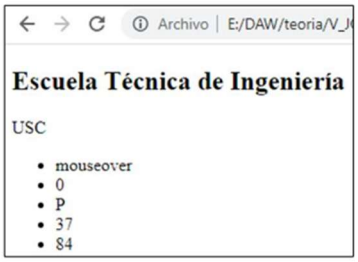
UI	focus, blur, change
Teclado	input, keydown, keyup, keypress
Ratón	click, dblclick, mouseup, mousedown, mouseover, mousemove, mouseout, hover
Formulario	submit, select, change
Documento	ready, load, unload
Navegador	error, resize, scroll

```
<h2>Escuela Técnica de Ingeniería</h2>
<p>USC</p>
<p id="info"></p>
```

```
<script>
   $(document).ready(function(){
     $("p").on('mouseover', function() {
        $("#info").text("Santiago de Compostela")
        });
   });
</script>
```

```
<h2>Escuela Técnica de Ingeniería</h2>
<p>USC</p>
<p id="info"></p>
```

```
<script>
   $(document).ready(function(){
     $("p").on('mouseover', function(e) {
        $("#info").text(e.type)
     });
   });
</script>
```

```
<h2>Escuela Técnica de Ingeniería</h2>
<p>USC</p>
<p id="info"></p>
```

```
<script>
   $(document).ready(function(){
     $("p").on('mouseover', function(e) {
        txt= "<ul><li>" + e.type + "</li>" +
        "<li>" + e.which + "</li>" +
        "<li>" + e.target.nodeName + "</li>" +
        "<li>" + e.pageX + "</li>" +
        "<li>" + e.pageY + "</li>" + "</u>"
        $("#info").html(txt)
     });
   });
</script>
```